



UNIT –III

Cognitive Models

Socio-organizational Issues And Stake Holder Requirements

By
Tamizharasi A
Assistant Professor
RMDEC

Cognitive Models

- Cognitive models represent users of interactive systems.
- They model aspects of user:
 - understanding
 - knowledge
 - intentions
 - processing



Cognitive Models

- Goal and Task Hierarchies
- Linguistic Model
- The Challenge of Display-Based Systems
- Physical and Device Models
- Cognitive Architectures

Goal and Task Hierarchies

- Many models make use of a model of mental processing in which the user achieves goals by solving subgoals in a divide-and-conquer
- Example: sales report

produce report

gather data

- find book names

- do keywords search of names database

- ... *further sub-goals*

- sift through names and abstracts by hand

- ... *further sub-goals*

- search sales database - further sub-goals

layout tables and histograms - further sub-goals

write description - further sub-goals

Issues for goal hierarchies

- Granularity
 - Where do we start?
 - Where do we stop?
- Routine learned behaviour, not problem solving
 - most abstract task is referred as unit task
- Conflict
 - More than one way to achieve a goal



Models / Techniques

1. *GOMS - Goals, Operators, Methods and Selection*
2. *CCT - Cognitive Complexity Theory*

GOMS

- *GOMS model of Card, Moran and Newell is an acronym for Goals, Operators, Methods & Selection*
- *Elements:*
 - Goals** - *user's goals describes what the user wants to achieve*
 - Operators**- *basic actions user performs*
 - Methods** - *decomposition of a goal into subgoals /operators*
 - Selection** - *means of choosing between competing methods*

Methods Example

- A selected window can be closed to an icon either by selecting the 'CLOSE' option or by hitting the 'L7' function key.
- In GOMS these two goal decompositions are referred to as methods

GOAL: CLOSE-WINDOW

```
.  [select GOAL: USE-MENU-METHOD
      .  MOVE-MOUSE-TO-FILE-MENU
      .  PULL-DOWN-FILE-MENU
      .  CLICK-OVER-CLOSE-OPTION
GOAL: USE-CTRL-W-METHOD
      .  PRESS-CONTROL-W-KEYS]
```

For a particular user:

Rule 1: Select USE-MENU-METHOD unless another rule applies

Rule 2: If the application is GAME,
select CTRL-W-METHOD

Selection

- *choosing between competing methods*

User Sam:

Rule 1: Use the CLOSE-METHOD unless another rule applies.

Rule 2: If the application is 'blocks' use the L7-METHOD.

- A typical GOMS analysis consist of a single high-level goal, which is then decomposed into a sequence of unit tasks, all of which can be further decomposed down to the level of basic operators.
- goal decomposition between the overall task and the unit tasks needs detailed understanding of the user's problem-solving strategies and of the application domain.

- 
- The original GOMS model serves as the basis for much of the cognitive modeling research in HCI.
 - good for describing how experts perform routine tasks.
 - Coupled with the physical device models, it can be used to predict the performance of the users in terms of execution times.

Cognitive Complexity Theory

- Kieras and Polson
- CCT has two parallel descriptions:
 1. Description of the user's goals in terms of production rules
 2. generalized transition networks

- 
- The production rules are a sequence of rules:

if condition then action

- *condition is a statement about the contents of working memory.*
- *If the condition is true then the production rule is said to fire.*
- *An action may consist of one or more elementary actions, which may be either changes to the working memory, or external actions such as keystrokes.*
- The production rule 'program' is written in a LISP-like language.

Example

Editing task using the UNIX vi text editor. The task is to insert a space where one has been missed out in the text. Say : 'cognitivecomplexity theory'.

(SELECT-INSERT-SPACE

IF (AND (TEST-GOAL perform unit task)
 (TEST-TEXT task is insert space)
 (NOT (TEST-GOAL insert space))
 (NOT (TEST-NOTE executing insert space)))
)

THEN ((ADD-GOAL insert space)
 (ADD-NOTE executing insert space)
 (LOOK-TEXT task is at %LINE
 %COL)))

(INSERT-SPACE-DONE

IF (AND (TEST-GOAL perform unit task)
 (TEST-NOTE executing insert space)
 (NOT (TEST-GOAL insert space)))
 THEN ((DELETE-NOTE executing insert
 space)
 (DELETE-GOAL perform unit task)
 (UNBIND %LINE %COL)))

(INSERT-SPACE-1

IF (AND (TEST-GOAL insert space)
 (NOT (TEST-GOAL move cursor))
 (NOT (TEST-CURSOR %LINE
 %COL))))

THEN ((ADD-GOAL move cursor to
 %LINE %COL)))

(INSERT-SPACE-2

IF (AND (TEST-GOAL insert space)
 (TEST-CURSOR %LINE %COL))
 THEN ((DO-KEYSTROKE 'I')
 (DO-KEYSTROKE SPACE)
 (DO-KEYSTROKE ESC)
 (DELETE-GOAL insert space)
))

- the contents of working memory (w.m.) are

(GOAL perform unit task)
(TEXT task is insert space)
(TEXT task is at 5 23)
(CURSOR 8 7)

- four rules are defined:

1. SELECT-INSERT-SPACE
2. INSERT-SPACE-DONE
3. INSERT-SPACE-1
4. INSERT-SPACE-2

only the first can fire

- The condition for SELECT-INSERT-SPACE is:

(AND (TEST-GOAL perform unit task)

true because (GOAL perform unit task) is in
w.m.

(TEST-TEXT task is insert space)

true because (TEXT task is insert space) is in
w.m.

(NOT (TEST-GOAL insert space))

true because (GOAL insert space) is *not in*
w.m.

(NOT (TEST-NOTE executing insert space)))

true because (NOTE executing insert space)
is *not* in w.m.

- The contents of working memory after the firing of rule SELECT-INSERT-SPACE are as follows:

(GOAL perform unit task)
(TEXT task is insert space)
(TEXT task is at 5 23)
(NOTE executing insert space)
(GOAL insert space)
(LINE 5)
(COL 23)
(CURSOR 8 7)

At this point since goal is changed, only rule INSERT-SPACE-1 can fire.



Problems and extensions of goal hierarchies

- The formation of a goal hierarchy is largely a post hoc technique and runs a very real risk of being defined by the computer dialog rather than the user
- the conceptual framework of goal hierarchies and user goal stacks can be used to express interface issues
- A general rule that can be applied to any goal hierarchy from this is that no higher-level goal should be satisfied until all subgoals have been satisfied.
- it is not always easy to predict when the user will consider a goal to have been satisfied.

Linguistic Model

- Understanding the user's behaviour and cognitive difficulty based on analysis of language between user and system.
- Similar in emphasis to dialogue models
- Backus–Naur Form (BNF)
- Task–Action Grammar (TAG)

Backus-Naur Form (BNF)

- To describe the dialog grammar
- A purely syntactic view of the dialogue
- Widely used to specify the syntax of computer programming languages
- many system dialogs can be described easily using BNF rules
- Terminals
 - lowest level of user behaviour
 - e.g. CLICK-MOUSE, MOVE-MOUSE
- Nonterminals
 - ordering of terminals
 - higher level of abstraction
 - e.g. select-menu, position-mouse

Example of BNF

- Basic syntax:
 - nonterminal ::= expression
- An expression
 - contains terminals and nonterminals
 - Use (+) for sequence and (|) choices
 - '::=' symbol is read as 'is defined as'

draw line ::= select line + choose points + last point
select line ::= pos mouse + CLICK MOUSE
choose points ::= choose one | choose one + choose points
choose one ::= pos mouse + CLICK MOUSE
last point ::= pos mouse + DBL CLICK MOUSE
pos mouse ::= NULL | MOVE MOUSE+ pos mouse

- The BNF description can be analyzed in various ways.

1. Count the number of rules. (The more rules an interface requires to use it, the more complicated it is.)

Eg: Choosepoints and choose-one with the single definition

**choose-points ::= position-mouse + CLICK-MOUSE |
position-mouse + CLICK-MOUSE + choose-points**

2. Counts the number of '+' and '|' operators

Note:

BNF ignore the advantages of consistency both in the language's structure and in its use of command names and letters.

Task–Action Grammar

- Uses parametrized grammar rules to emphasize consistency and encoding the user's world knowledge
- Eg:

3 UNIX commands:

cp (for copying files),
mv (for moving files) and
ln (for linking files).

Each of these has two possible forms. They either have two arguments, a source and destination filename, or have any number of source filenames followed by a destination directory:

Consistency in TAG

- In BNF, three UNIX commands would be described as:

copy ::= cp + filename + filename | cp + filenames + directory

move ::= mv + filename + filename | mv + filenames + directory

link ::= ln + filename + filename | ln + filenames + directory

- No BNF measure could distinguish between this and a less consistent grammar in which

link ::= ln + filename + filename | ln + directory + filenames

Consistency in TAG (cont'd)

- consistency of argument order made explicit using a parameter, or semantic feature for file operations

- Feature Possible values

Op = copy; move; link

- Rules

file-op[Op] ::= command[Op] + filename + filename
 | command[Op] + filenames + directory

command[Op = copy] ::= cp

command[Op = move] ::= mv

command[Op = link] ::= ln

- TAG has features for talking about 'world knowledge'.
- For example, imagine we have two command line interfaces for moving a mechanical turtle around the

Command interface 1

```
movement [Direction]
    := command[Direction] + distance + RETURN
command[Direction=forward]    := 'go 395'
command[Direction=backward]   := 'go 013'
command[Direction=left]       := 'go 712'
command[Direction=right]      := 'go 956'
```

Command interface 2

```
movement [Direction]
    := command[Direction] + distance + RETURN
command[Direction=forward]    := 'FORWARD'
command[Direction=backward]   := 'BACKWARD'
command[Direction=left]       := 'LEFT'
command[Direction=right]      := 'RIGHT'
```

- command 'go 395' refers to the address of a machine-code routine, which performs the appropriate movement.
- second interface is preferable to the first.
- TAG includes a special form known-item, which is used to denote information that the user will already

TAG rules for second interface

Command interface 2

movement [Direction]

:= command[Direction] + distance + RETURN

command[Direction] := known-item[Type=word,Direction]

* command[Direction=forward] := 'FORWARD'

* command[Direction=backward] := 'BACKWARD'

* command[Direction=left] := 'LEFT'

* command[Direction=right] := 'RIGHT'

Other uses of TAG

- User's existing knowledge
- Congruence between features and commands
- These are modelled as derived rules

Physical and device models

- The Keystroke Level Model (KLM)
- Buxton's 3-state model

The Keystroke Level Model (KLM)

- Uses human motor system *understanding as a basis for detailed predictions about user performance.*
- aimed at unit tasks within interaction – the execution of simple command sequences, typically taking no more than 20 seconds
- Eg:
 - search and replace feature, or changing the font of a word
- more complex tasks would be split into subtasks before the user attempts to map them into physical actions.
- The task is split into two phases:
 1. **acquisition of the task, when the user builds a mental representation of the task;**
 2. **execution of the task using the system's facilities.**

- 
- During the acquisition phase, the user will have decided how to accomplish the task using the primitives of the system
 - So during the execution phase, there is no high-level mental activity – the user is effectively expert.
 - The model decomposes the execution phase into five different physical motor operators, a mental operator and a system response operator:
 - **K** Keystroking, actually striking keys, including shifts and other modifier keys.
 - **B** Pressing a mouse button.
 - **P** Pointing, moving the mouse (or similar device) at a target.
 - **H** Homing, switching the hand between mouse and keyboard.
 - **D** Drawing lines using the mouse.
 - **M** Mentally preparing for a physical action.

- The execution of a task will involve interleaved occurrences of the various operators.
- Eg: using a mouse-based editor. If we notice a single character error we will point at the error, delete the character and retype it, and then return to our previous typing point.
- This is decomposed as follows:
 1. Move hand to mouse **H[mouse]**
 2. Position mouse after bad character **PB[LEFT]**
 3. Return to keyboard **H[keyboard]**
 4. Delete character **MK[DELETE]**
 5. Type correction **K[char]**
 6. Reposition insertion point **H[mouse]MPB[LEFT]**

- The model predicts the total time taken during the execution phase by adding the component times for each of the above activities.
- Eg: if the time taken for one keystroke is t_K , then the total time doing keystrokes is

$$TK = 2t_K$$

- Similar calculations for the rest of the operators give a total time of

$$T_{execute} = TK + TB + TP + TH + TD + TM + TR$$

Table 12.1 Times for various operators in the keystroke-level model (adapted from Card, Moran and Newell [56], published and reprinted by permission of Lawrence Erlbaum Associates, Inc.)

Operator	Remarks	Time (s)
K	Press key	
	good typist (90 wpm)	0.12
	poor typist (40 wpm)	0.28
	non-typist	1.20
B	Mouse button press	
	down or up	0.10
	click	0.20
P	Point with mouse	
	Fitts' law	$0.1 \log_2(D/S + 0.5)$
	average movement	1.10
H	Home hands to and from keyboard	0.40
D	Drawing – domain dependent	–
M	Mentally prepare	1.35
R	Response from system – measure	–

wpm = words per minute

- The keying time depends on the typing skill of the user
- Pressing a mouse is usually quicker
- pointing time is calculated using Fitts' law (depends on the size and position of the target)
- Drawing time depends on the number and length of the lines drawn
- homing time and mental preparation time are assumed constant.

- 
- The physical operator times all depend on the skills of the user.
 - Also, the mental operator depends on the level of chunking, and hence the expertise of the user
 - predictions made by KLM are only meant to be an approximation
 - But capable of giving accurate quantitative predictions about performance.

KLM example

GOAL: ICONISE-WINDOW

[select

GOAL: USE-CLOSE-METHOD

. MOVE-MOUSE-TO- FILE-MENU

. PULL-DOWN-FILE-MENU

. CLICK-OVER-CLOSE-OPTION

GOAL: USE-CTRL-W-METHOD

PRESS-CONTROL-W-KEY]

- compare alternatives:

- USE-CTRL-W-METHOD vs.
- USE-CLOSE-METHOD

- assume hand starts on mouse

USE-CTRL-W-METHOD		USE-CLOSE-METHOD	
H[to kbd]	0.40	P[to menu]	1.1
M	1.35	B[LEFT down]	0.1
K[ctrlW key]	0.28	M	1.35
		P[to option]	1.1
		B[LEFT up]	0.1
Total	2.03 s	Total	3.75 s

Three-state model

- captures the crucial distinctions among the various pointing devices like mouse, light pen etc.



Mouse transitions: states 1 and 2

- Moving the mouse with no buttons pushed, normally moves the mouse cursor about.
- This tracking behavior is termed **state 1**.
- Depressing a button over an icon and then moving the mouse will often result in an object being dragged about.
- This is termed **state 2**

- consider a light pen with a button, it behaves just like a mouse when it is touching the screen.
- When its button is not depressed, it is in **state 1**
- When its button is down, **state 2**.
- light pen has a third state, when the light pen is not touching the screen. In this state the system cannot track the light pen's position. This is called **state 0**

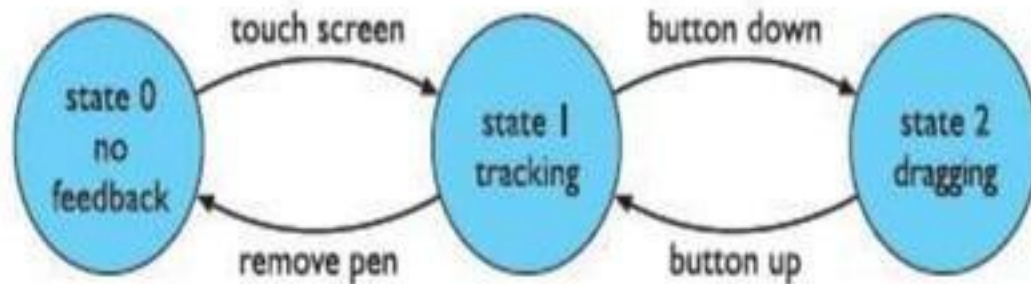


Figure 12.2 Light pen transitions: three states

- Fitts' law has different timing constants for different devices.
- Recall that Fitts' law says that the time taken to move to a target of size S at a distance D is:
 - $a + b \log_2(D/S + 1)$
- The constants a and b depend on the particular pointing device used and the skill of the user with that device. also depend on the device state

Table 12.2 Fitts' law coefficients (after MacKenzie, Sellen and Buxton [221], © 1991 ACM, Inc. Reprinted by permission)

	Device	a (ms)	b (ms/bit)
Pointing (state 1)	Mouse	-107	223
	Trackball	75	300
Dragging (state 2)	Mouse	135	249
	Trackball	-349	688

KLM prediction for the CLOSE-METHOD using these data.

- Recall that the method had two pointing operators, one to point to the window's title bar (with a distance to target size ratio of 10:1), the second to drag the selection down to 'CLOSE' on the pop-up menu (4:1).
- Thus the first pointing operator is state 1 and the second is state 2.
- The times are thus
- Mouse
 - $P[\text{to menu bar}] = -107 + 223 \log_2(11) = 664 \text{ ms}$
 - $P[\text{to option}] = 135 + 249 \log_2(5) = 713 \text{ ms}$
- Trackball
 - $P[\text{to menu bar}] = 75 + 300 \log_2(11) = 1113 \text{ ms}$
 - $P[\text{to option}] = -349 + 688 \log_2(5) = 1248 \text{ ms}$



COGNITIVE ARCHITECTURES

- for the architectural models the prediction and understanding of error is central to their analyses.

2 Architectural Models:

- The problem space model
- Interacting cognitive subsystems

The problem space model

- In computer science, it is common to describe a problem as the search through a set of possible states, from some initial state to a desired state.
- The search proceeds by moving from one state to another possible state by means of operations or actions, the ultimate goal of which is to arrive at one of the desired states.

- 
- The architecture of the machine only allows the definition of the search or *problem space* and the actions that can occur to traverse that space.
 - Termination is also assumed to happen once the desired state is reached.
 - machine does not have the ability to formulate the problem space and its solution, because it has no idea of the goal.
 - It is the job of the programmer to understand the goal and so define the machine to

- 
- *problem space model, based on the problem-solving work of Newell and Simon at Carnegie–Mellon University*
 - A problem space consists of a set of states and a set of operations that can be performed on the states.
 - Behavior in a problem space is a two-step process.
 1. the current operator is chosen based on the current

- 
- The problem space must represent rational behavior i.e achieve the goal
 - A problem space represents a goal by defining the desired states as a subset of all possible states.
 - Once the initial state is set, the task within the problem space is to find a sequence of operations that form a path within the state space from the initial state to one of the desired states, whereupon successful termination occurs.

- 
- four different activities that occur within a problem space:

1. goal formulation
2. operation selection,
3. operation application
4. goal completion.

Goal Formulation

- creates the initial state based on observations of the external environment.
- Operations are the actions at the knowledge level in the problem space

Operation selection

- selects the appropriate operation at a given point in time.
- Suggests an operation that can act on that state and transform it 'closer' to a desired state.



Operation application

- executes the operation, changing the current state and surrounding environment.

Goal Completion

- If the new state is a desired state, then the goal has been achieved and the goal completion process reverts the agent to inactive.
- The real power of the problem space architecture is in recursion

Problem space model is the basis for SOAR cognitive architecture

Interacting cognitive subsystems

- ICS provides a model of perception, cognition and action
- It is not intended to produce a description of the user in terms of sequences of actions that he performs.
- ICS provides a more holistic view of the user as an information-processing machine.
- The emphasis is on determining how easy particular procedures of action sequences become as they are made more automatic within the user

- 
- incorporate two separate psychological traditions within one cognitive architecture.
1. the architectural and general-purpose information-processing approach of short-term memory research.
 2. computational and representational approach characteristic of psycholinguistic research and AI problem-solving literature.

- 
- architecture of ICS is the coordinated activity of nine smaller subsystems:
 - five peripheral subsystems are in contact with the physical world
 - four are central, dealing with mental processes.
 - Each subsystem has the same generic structure.
 - A subsystem is described in terms of its typed inputs and outputs along with a memory store for holding typed information.

- 
- Each of the nine subsystems is specialized for handling some aspect of external or internal processing
 - Eg:
 - Visual system for describing what is seen in the world.
 - processing of propositional information, capturing the attributes and identities of entities and their relationships with each other
 - ability to explain how a user proceduralizes action
 - ICS has been suggested as a design tool that can act as an expert system to advise a designer in developing an interface.